UADE

APLICACIONES INTERACTIVAS

Cuatrimestre 2do de 2026

TRABAJO PRÁCTICO

Ecosistema DeliveryGo – Módulo de Pagos

DOCENTE

Alejandro Foglino

Curso: Martes, Noche

EQUIPO Nº: 03

1182205	Diaz, Federico
1155256	Gómez Traverso, Juliana
1155385	Carpiniello, Santino
1129290	Liuboschitz, Tobias

Fecha de presentación: 22/09/2026

Observaciones:

……

……

……

……

Introducción y contexto del módulo

DeliveryGo es una plataforma argentina ficticia de pedidos y entregas a domicilio que está dejando de ser una aplicación monolítica para convertirse en un ecosistema de servicios independientes, integrados entre sí mediante APIs REST. El Grupo 3 quedó a cargo del Módulo de Pagos. Nuestra función dentro del ecosistema es registrar los intentos de pago asociados a los pedidos, autorizarlos o rechazarlos, y gestionar devoluciones simuladas, sin procesar dinero real ni conectarnos a ninguna pasarela bancaria real.

Este documento corresponde a la Entrega 1 del TPO DeliveryGo, la etapa de análisis y arquitectura. El objetivo es dejar aprobado el diseño del módulo antes de empezar a programar, así que acá presentamos el alcance, los actores involucrados, las reglas de negocio, los casos de uso, los diagramas C4, el diagrama de clases en UML, el Diagrama Entidad Relación, la forma en que vamos a persistir los datos con JPA, cómo pensamos manejar los errores y el backlog con el que arrancamos.

Objetivo del módulo

Que un pedido confirmado pueda pagarse: registrar el intento de pago, autorizarlo o rechazarlo, evitar que un mismo pago se duplique si la solicitud llega dos veces, permitir consultar en qué estado está y poder reembolsarlo cuando corresponda. Además, avisar a los módulos de Pedidos, Usuarios y Notificaciones sobre lo que va pasando con cada pago.

Alcance y actores

Alcance

El módulo de Pagos se encarga de lo siguiente:

- Recibir la solicitud de pago que envía el Servicio de Pedidos, con el monto y el medio de pago elegidos por el cliente.
- Verificar que el importe coincida con el total informado por Pedidos y evita que una misma solicitud genere dos pagos.
- Simular la autorización o rechazo del pago, ya que no hay conexión a ninguna pasarela bancaria real.
- Guarda un historial de los cambios de estado de cada pago, para poder reconstruir qué pasó.
- Gestiona reembolsos totales o parciales sobre pagos que ya fueron aprobados.
- Expone una API REST documentada con OpenAPI y Swagger para que el Portal Web y los demás módulos puedan consumirla.
- Avisa al Servicio de Notificaciones cuando ocurre algo relevante, como una aprobación o un rechazo.

Fuera de alcance

- Procesar dinero real o integrarse con pasarelas de pago reales, como Mercado Pago o tarjetas verdaderas.
- Guardar datos sensibles reales de tarjetas. Solo se almacenan datos simulados y enmascarados.
- Conciliación bancaria o facturación electrónica.

Actores

Actor	Rol frente al módulo de Pagos
Cliente (a través del Portal Web)	Es quien, al confirmar su pedido, origina de forma indirecta la solicitud de pago y luego consulta el estado de ese pago.
Servicio de Pedidos	Pide la creación del pago, recibe el resultado de la autorización y puede disparar un reembolso si el pedido se cancela.
Servicio de Usuarios y Autenticación	Permite validar la identidad y el estado del usuario que está pagando, mediante una consulta por su código de usuario.
Servicio de Notificaciones	Recibe la solicitud de enviar comprobantes y avisos de rechazo o reembolso.
Administrador	Consulta pagos y reembolsos con fines de auditoría o para resolver incidencias.
Analista	Consume información agregada de pagos para las métricas del negocio, a través del módulo de Analítica.

Reglas de negocio

Estas son las reglas que definimos para el módulo. Las primeras tres son las que pide la consigna del TPO de forma explícita y el resto las agregamos nosotros para cubrir el ciclo completo del pago.

1. Si llega dos veces la misma solicitud de pago, con la misma clave de control, no se debe crear un segundo pago: se devuelve el que ya existe.
2. El importe del pago tiene que coincidir exactamente con el total que informa el Servicio de Pedidos.
3. Solo un pago en estado APROBADO permite que el pedido pase a la siguiente etapa.
4. Las transiciones de estado válidas son PENDIENTE a APROBADO, PENDIENTE a RECHAZADO y APROBADO a REEMBOLSADO. Cualquier otro cambio de estado se considera inválido.
5. Un pago RECHAZADO no puede reintentarse con la misma clave de control. Para reintentar hace falta una clave nueva que haga referencia al mismo pedido.
6. Un reembolso solo puede pedirse sobre un pago que esté en estado APROBADO.
7. La suma de los reembolsos de un pago no puede superar el monto que fue aprobado.
8. No se guardan datos sensibles reales de los medios de pago, solo un detalle enmascarado, por ejemplo, los últimos 4 dígitos de una tarjeta.
9. Cada cambio de estado de un pago queda registrado en el historial, con el estado anterior, el estado nuevo, el motivo y la fecha.
10. Repetir una operación que ya se aplicó, como autorizar, rechazar o reembolsar, no genera efectos adicionales: el sistema revisa el estado actual antes de aplicar cualquier cambio.

Casos de uso

Incluimos los cinco casos de uso mínimos y centrales que pide la consigna (iniciar pago, autorizar o rechazar, consultar y reembolsar) y sumamos cinco casos de uso no tan centrales para completar el ciclo funcional del módulo.

CU-01. Iniciar pago

ID	CU-01
Nombre	Iniciar pago
Actor principal	Servicio de pedidos
Objetivo	Registrar intento de pago valido y unico para un pedido
Precondiciones	Que el pedido exista y este en estado CONFIRMADO, y tanto el usuario como el medio de pago son válidos.
Flujo principal	1. Pedidos envía un POST a /api/v1/payments con el código de pedido, el código de usuario, el monto, el medio de pago y una clave de control. 2. Pagos valida al usuario consultando al Servicio de Usuarios. 3. Pagos revisa si esa clave de control ya se usó antes. Si ya existe, devuelve el pago original en lugar de crear uno nuevo. 4. Pagos verifica que el monto coincida con el que le solicito a Pedidos. 5. Pagos crea el registro en estado PENDIENTE y responde 201 Created con el código del pago.
Flujo alternativo	*3a. Si la clave de control ya se había usado, responde 200 OK con el pago existente.* *4a. Si el monto no coincide, responde 422 PAYMENT_AMOUNT_MISMATCH*
Resultado	Queda creado un pago en estado PENDIENTE, listo para ser autorizado.
RN Asociados	RN-02, RN-05, RN-08
Entidades/ relaciones	ENT-01 Pago, ENT-03 MedioPago / REL-02
Secuencia	SEC-01
Servicio	SRV-01 iniciarPago()
API/ DTO	API-01 POST /api/v1/payments / DTO-01 - DTO-02

CU-02. Autorizar pago

ID	CU-02
Nombre	Autorizar pago
Actor principal	Servicio de Pagos
Objetivo	Aprobar pago pendiente mediante autorizacion simulada
Precondiciones	El actor está autenticado y cuenta con el código del pago que quiere consultar.
Flujo principal	1. El actor envía POST /api/v1/payments/{codigoPago}/approve 2. El sistema busca el pago por su codigo 3. El sistema verifica si el pago tiene estado PENDIENTE. 4. El estado pasa a APROBADO y se avisa a Pedidos.

	5. Se registra el cambio en el historial de estados. 6. Se le pide al Servicio de Notificaciones que envíe el comprobante de pago.
Flujo alternativo	*3a. Si el pago no está en estado PENDIENTE, responde 409 PAYMENT_INVALID_STATE.*
Resultado	El pago queda en estado APROBADO, con el historial actualizado.
RN Asociados	RN-03, RN-04, RN-09, RN-10
Entidades/ relaciones	ENT-01, ENT-04 HistorialEstadoPago / REL-03
Secuencia	SEC-02
Servicio	SRV-02 aprobarPago()
API/ DTO	API-02 POST /api/v1/payments/{codigoPago}/approve / DTO-02

CU-03. Rechazar pago

ID	CU-03
Nombre	Rechazar pago
Actor principal	Servicio de Pagos (proceso interno simulado)
Objetivo	Rechazar un pago pendiente
Precondiciones	El actor está autenticado y cuenta con el código del pago que quiere consultar.
Flujo principal	1. El actor envía POST /api/v1/payments/{codigoPago}/reject 2. El sistema busca el pago por su código 3. Ell sistema verifica si el pago tiene estado PENDIENTE. 4. El pago se rechaza, el estado pasa a RECHAZADO y se avisa a Pedidos. 5. Se registra el cambio en el historial de estados. 6. Se le pide al Servicio de Notificaciones que envíe el aviso de rechazo.
Flujo alternativo	*3a. S*i el pago ya está APROBADO, responde 200 con el pago sin modificarlo; si está en otro estado, responde 409. *Si el pago no está en estado PENDIENTE, responde 409 PAYMENT_INVALID_STATE.*
Resultado	El pago queda en estado RECHAZADO, con el historial actualizado.
RN Asociados	RN-04, RN-05, RN-09, RN-10
Entidades/ relaciones	ENT-01, ENT-04 / REL-03
Secuencia	SEC-03
Servicio	SRV-03 rechazarPago()
API/ DTO	API-03 POST /api/v1/payments/{codigoPago}/reject / DTO-03 - DTO-02

CU-04. Consultar pago

ID	CU-04
Nombre	Consultar Pago
Actor principal	Portal Web, Servicio de Pedidos o Administrador

Objetivo	Permitir consultar el estado actual, monto y medio de pago de un pago por su código público.
Precondiciones	El actor está autenticado y cuenta con el código del pago que quiere consultar.
Flujo principal	1. El interesado hace un GET a /api/v1/payments/{codigoPago}. 2. Pagos busca el registro por su código público, nunca por el identificador interno. 3. Pagos responde con el estado actual, el monto y el medio de pago utilizado.
Flujo alternativo	*2a. Si el código de pago no existe, responde 404 PAYMENT_NOT_FOUND.*
Resultado	Quien consulta obtiene el estado actualizado del pago.
RN Asociados	RN-08
Entidades/ relaciones	ENT-01, ENT-03 / REL-02
Secuencia	SEC-04
Servicio	SRV-04 consultarPago()
API/ DTO	API-04 GET /api/v1/payments/{codigoPago} / DTO-02

CU-05. Reembolsar pago

ID	CU-05
Nombre	Reembolsar pago
Actor principal	Servicio de Pedidos o Administrador
Objetivo	Devolver total o parcialmente un pago aprobado
Precondiciones	El actor está autenticado y cuenta con el código del pago que quiere consultar.
Flujo principal	1. Se envía un POST a /api/v1/payments/{codigoPago}/refund con el monto y el motivo del reembolso. 2. Pagos valida que exista un pago con dicho codigo. 3. Pagos verifica que el estado del pago encontrado sea APROBADO. 4. Pagos valida que el monto solicitado no supere lo que queda disponible para reembolsar.

	5. Se crea el registro de Reembolso en estado APROBADO, de forma simulada, y si el reembolso es total el pago pasa a estado REEMBOLSADO. 6. Se genera el comprobante correspondiente. 7. Se avisa al usuario a través del Servicio de Notificaciones.
Flujo alternativo	*2a. Pago inexistente, responde 404.* *3a. Si el pago no está aprobado, responde 409 PAYMENT_INVALID_STATE.* *3a. Si el monto de reembolso supera lo disponible, responde 422 REFUND_EXCEEDS_AMOUNT.*
Resultado	El pago queda marcado como reembolsado, total o parcialmente, y se genera el comprobante correspondiente.
RN Asociados	RN-04, RN-06, RN-07, RN-09, RN-10
Entidades/ relaciones	ENT-01, ENT-02 Reembolso, ENT-04 / REL-01, REL-03
Secuencia	SEC-05
Servicio	SRV-05 reembolsarPago()
API/ DTO	API-05 POST /api/v1/payments/{codigoPago}/refund / DTO-04 - DTO-05

CU-06. Listar pagos

ID	CU-06
Nombre	Listar pagos por usuario
Actor principal	Administrador o analista
Objetivo	Obtener un listado paginado y filtrable de pagos para auditoría y métricas.
Precondiciones	Quien consulta está autenticado y tiene los permisos necesarios
Flujo principal	1. Se envía un GET a /api/v1/payments con filtros opcionales, como estado, rango de fechas o código de usuario, y paginación con page, size y sort. 2. Pagos consulta el repositorio aplicando esos filtros. 3. Pagos responde con la página de resultados que corresponde.
Flujo alternativo	*2a. Si los parámetros de filtro no son válidos, responde 400 BAD_REQUEST.*
Resultado	Se obtiene un listado paginado de pagos según los filtros aplicados.
RN Asociados	RN-08
Entidades/ relaciones	
Secuencia	
Servicio	
API/ DTO	

CU-07. Consultar pago asociado a un pedido

ID	CU-07
Nombre	Consultar pago asociado a un pedido
Actor principal	Administrador
Objetivo	Recuperar los pagos asociados a un pedido para reconstruir su trazabilidad.
Precondiciones	El pago fue realizado
Flujo principal	1. Se envía un GET a /api/v1/payments/{idPedido}. 2. Pagos consulta por el pago requerido a tal pedido. 3. Pagos recupera el pago por el pedido solicitado.
Flujo alternativo	*2a. Si el código de pago no existe, responde 404 PAYMENT_NOT_FOUND.*
Resultado	Se obtiene la trazabilidad del pago
RN Asociados	RN-05
Entidades/ relaciones	
Secuencia	
Servicio	
API/ DTO	

CU-08. Administrar medios de pago

ID	CU-08
Nombre	Administrar medios de pago
Actor principal	Cliente, a través del Portal Web
Objetivo	Registrar un medio de pago simulado del usuario guardando solo un detalle enmascarado.
Precondiciones	El usuario está autenticado.
Flujo principal	1. El cliente envía los datos del medio de pago que quiere usar, con su tipo y detalle. 2. Pagos enmascara el detalle sensible antes de guardarlo. 3. Pagos crea el registro de MedioPago asociado al usuario y responde con su código.
Flujo alternativo	*1a. Si el tipo de medio de pago no es soportado, responde 400 PAYMENT_METHOD_INVALID*
Resultado	El medio de pago queda disponible para usarse en futuros pagos.

RN Asociados	RN-08
Entidades/ relaciones	
Secuencia	
Servicio	
API/ DTO	

CU-09. Detectar solicitudes duplicadas mediante idempotencia.

ID	CU-09
Nombre	Detectar solicitudes duplicadas mediante idempotencia
Actor principal	Servicio de pedidos
Objetivo	Evitar que una misma solicitud de pago genere más de un pago, y detectar reutilización de la clave con datos distintos.
Precondiciones	La solicitud de pago incluye clave idempotente
Flujo principal	1. Se envía un POST /api/v1/payments(claveIdemUnica) 2. Pagos consulta si existe ese pago. 3. Si no existe, lo crea. 4. Si ya existe, devuelve el pago original.
Flujo alternativo	*2.a Pagos detecta que la clave fue usada en un pago pero la solicitud trae datos diferentes. Rechaza la solicitud y responde 409 DUPLICATE_PAYMENT_REQUEST*
Resultado	Se evita la creación de solicitudes duplicadas.
RN Asociados	RN-01
Entidades/ relaciones	
Secuencia	
Servicio	
API/ DTO	

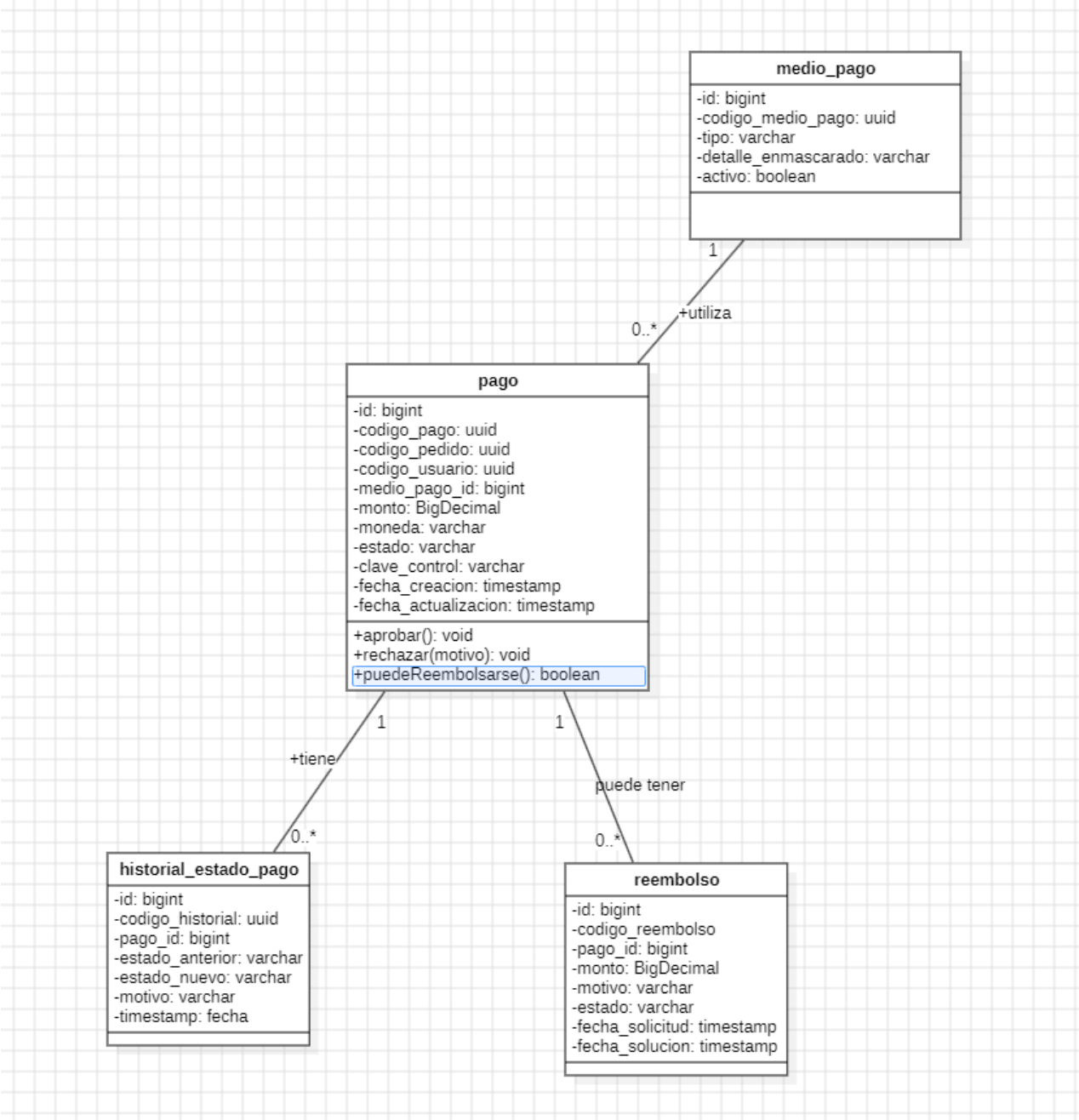
CU-10. Registrar historial de estados del pago

ID	CU-10
Nombre	Registrar historial de estados de pago
Actor principal	Administrador
Objetivo	Consultar la línea de tiempo de cambios de estado de un pago.
Precondiciones	El pago existe.
Flujo principal	1. Se envía un GET a /api/v1/payments/{codigoPago}/history.

	2. Pagos recupera todos los registros de HistorialEstadoPago asociados, ordenados por fecha. 3. Pagos responde con la línea de tiempo completa del pago.
Flujo alternativo	*2.a Si el código de pago no existe, responde 404 PAYMENT_NOT_FOUND.*
Resultado	Se obtiene la trazabilidad completa del pago, útil para auditoría.
RN Asociados	RN-09
Entidades/ relaciones	
Secuencia	
Servicio	
API/ DTO	

Arquitectura del módulo

Diagrama de Clases (UML)



Reglas de negocio - Tabla

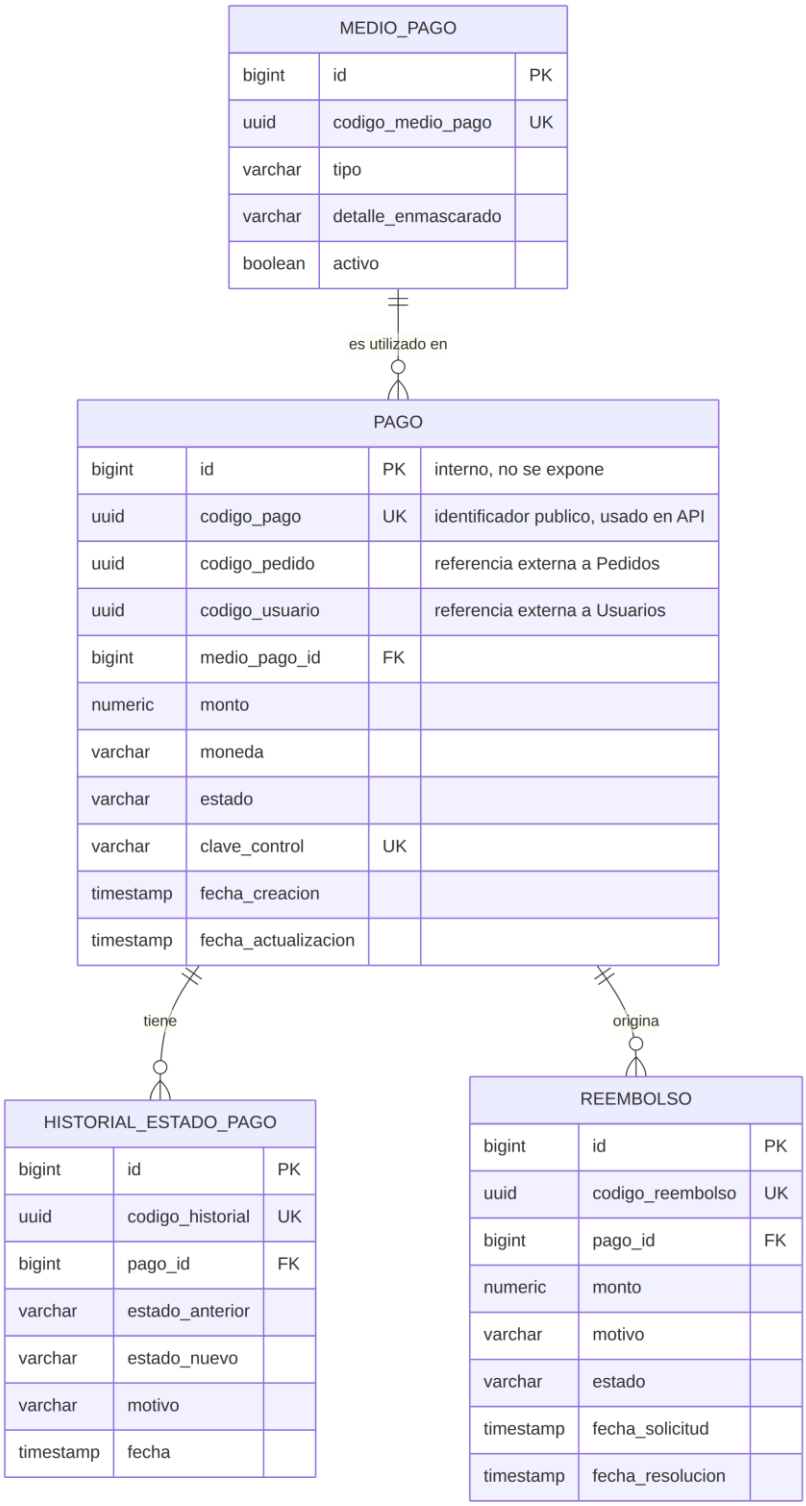
ID	Regla	CU Relacionados	Donde se implementa
RN-01	Si llega dos veces la misma solicitud de pago con la misma clave de control, no se crea un segundo pago: se devuelve el	**CU-01**	SRV-01 / REP-01 (restricción UNIQUE sobre clave_control)

	existente. Si la clave ya se usó con datos distintos, se rechaza la solicitud.		
RN-02	El importe del pago debe coincidir exactamente con el total del pedido informado por el Servicio de Pedidos.	**CU-01**	SRV-01 / INT-01
RN-03	Solo un pago en estado APROBADO permite que el pedido avance a la siguiente etapa.	**CU-02,CU-03**	SRV-02, SRV-03 / INT-04
RN-04	Las transiciones de estado válidas son PENDIENTE → APROBADO, PENDIENTE → RECHAZADO y APROBADO → REEMBOLSADO. Cualquier otra transición es inválida.	**CU-02, CU-03, CU-05**	SRV-02, SRV-03, SRV-05 / entidad Pago (ENT)
RN-05	Un pago RECHAZADO no puede reintentarse con la misma clave de control; el reintento requiere una clave nueva que referencie al mismo pedido.	**CU-01, CU-03**	SRV-01 / REP-01
RN-06	Solo puede reembolsarse un pago en estado APROBADO.	**CU-05**	SRV-05 / REP-02
RN-07	La suma de los reembolsos de un pago no puede superar el monto aprobado.	**CU-05**	SRV-05 / REP-03
RN-08	No se almacenan datos sensibles reales del medio de pago; solo un detalle enmascarado (p. ej. últimos 4 dígitos).	**CU-01, CU-04**	SRV-01 / entidad MedioPago (ENT)

RN-09	Cada cambio de estado de un pago se registra en el historial con estado anterior, estado nuevo, motivo y fecha.	**CU-02, CU-03, CU-05**	SRV-02, SRV-03, SRV-05 / REP-04
RN-10		**CU-02, CU-03, CU-05**	SRV-02, SRV-03, SRV-05

Diagrama de Entidad – Relación (DER)

Cada tabla tiene un id interno autoincremental, de tipo bigint, que solo se usa para relaciones internas y búsquedas más rápidas, y una columna código, de tipo uuid con restricción UNIQUE, que es la que se expone en la API y en las referencias hacia otros módulos, como codigo_pedido o codigo_usuario.



Manejo de errores

Campo	Descripción
timestamp	Fecha y hora del error, en formato ISO-8601.
status	Código HTTP asociado al error.
code	Código de error propio del módulo, según el listado de la tabla 9.1.
message	Mensaje descriptivo pensado para quien consume la API.
path	Endpoint donde ocurrió el error.
correlationId	Identificador de correlación, tomado del header X-Correlation-Id, que sirve para seguir el rastro entre módulos.

Códigos de error del módulo

Código	HTTP	Situación
PAYMENT_NOT_FOUND	404	No existe un pago con el código informado.
PAYMENT_AMOUNT_MISMATCH	422	El monto del pago no coincide con el total del pedido.
PAYMENT_INVALID_STATE	409	Se intenta una operación que no está permitida para el estado actual del pago.
DUPLICATE_PAYMENT_REQUEST	409	La clave de control ya se usó antes con datos distintos.
REFUND_EXCEEDS_AMOUNT	422	El monto de reembolso solicitado supera lo disponible.
PAYMENT_METHOD_INVALID	400	El tipo o los datos del medio de pago no son válidos.
REFUND_NOT_FOUND	404	No existe un reembolso con el código informado.